

Team members: Ameer Aldarraj, Bjorn Addington, Nelson Bustos Nuno, Thomas Velleff

Team BANT Design Document



Project description:

For this project, our team decided to create a text-based game. The database we would use would reflect the interactions the player would have with the game throughout the save state they are using. In game things, such as items or currency, would be saved in the database. As a player progresses throughout the game, they would interact with the different characters in the world and proceed on different small quest lines. The GUI would consist of the game being played, an options bar, and a section of the screen for the dialog window.

To start, there will be three empty saves available to the player. Once an empty save is used, they will enter said save and be allowed to progress throughout the game. However, if the player tries to use a different already used save, they will need to enter a password to play it. This enables a dynamic way for security and to ensure each player can play their own save. If a player goes to exit the game, they will need to save first and if there is not already a password in place for that save such as in the scenario they are using an empty save, they will need to set one.

Over the course of the game, the player must uncover the true objective, going through basic combat scenarios, solving puzzles, and making the correct choices to win.

Overall, this game leads inspiration from OpenWorld settings however, in the interest of ability we will constrain the game to a set of objectives that implement one goal. Then, if we have time, we will add more. Our project takes an OOP approach to development,

breaking out our application in 14 current files and a number of other technological dependencies.

User Guide:

Required Applications/files:

This project was built through IntelliJ Idea and uses JavaFX for the frontend, as well as SQLite for the database. To run this game, you will have to have the required JavaFX and SQLite files included in the project directory.

From Startup Screen:

Once in the application itself, the user will be prompted to enter a password in one of three different save slots.

SaveSlotOne: "Password1"

SaveSlotTwo: "Password2"

SaveSlotThree: "Password3"

When a password is entered, it will take you to the MAP scene where you will see a physical map with various buttons. The buttons below the map will be to check your inventory and send you back to the menu. The red buttons on the map will take you to the different types of gameplays, the blue button will take you to the shop, the gray button will prompt you to open the door with a key, and the yellow button will let you know what you need to finish the game and if you have met those conditions, it will take you to the end screen.

Shop scene:

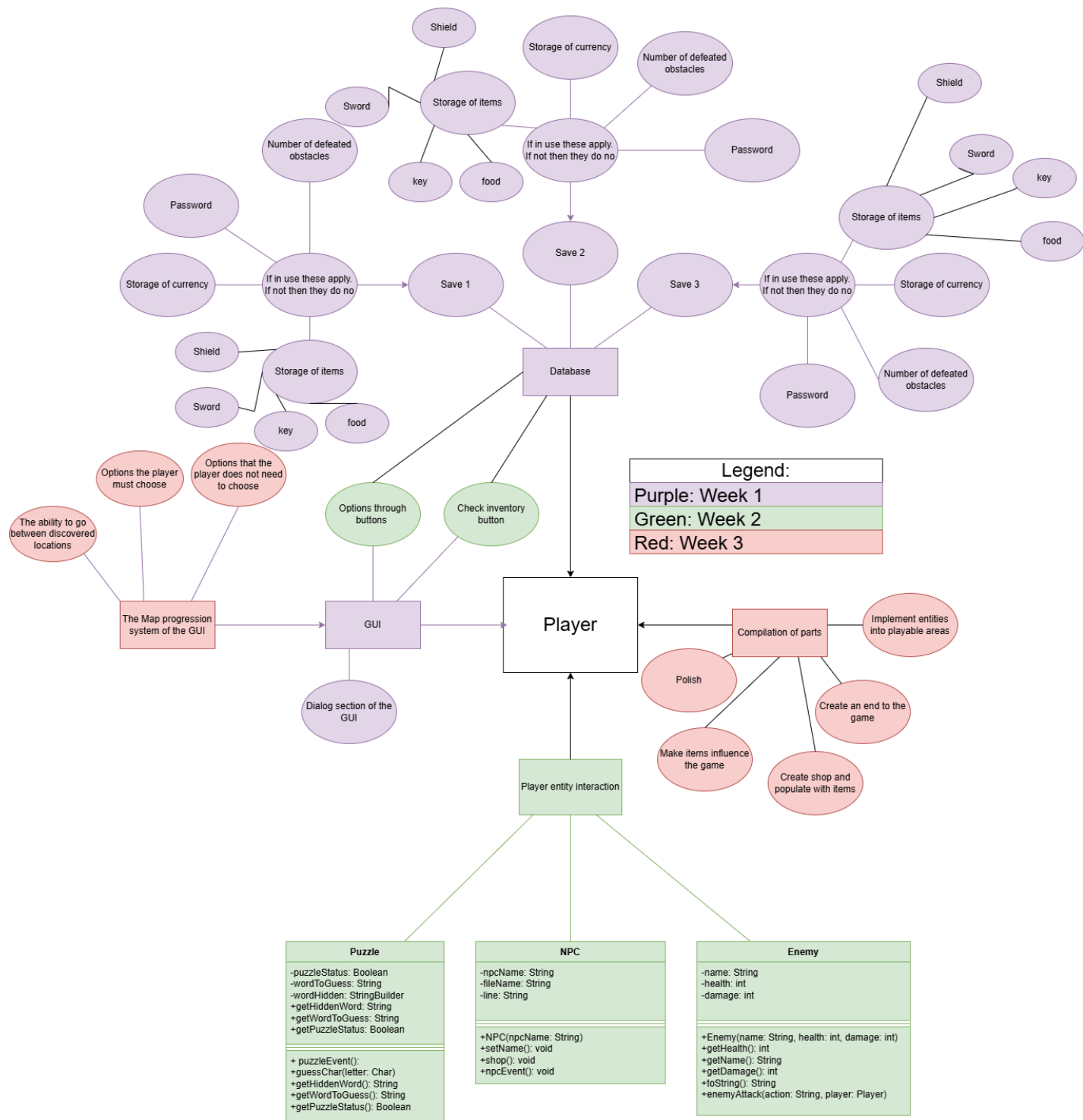
The shop scene will present you with a button to check your inventory, eat food, and buy or sell items. When buying and selling items, you will see the items appear and disappear from your inventory in the text box.

Gameplay Battle/Puzzle screen:

When in the battle screen you will be prompted to press one of four buttons each turn. The buttons are attack flee, eat food, and shield. When attacking you will have a chance to hit the enemy and if you succeed you will see the enemy health pool go down as a result, eat food will heal you, shield will block incoming attacks, and fleeing will exit the battle.

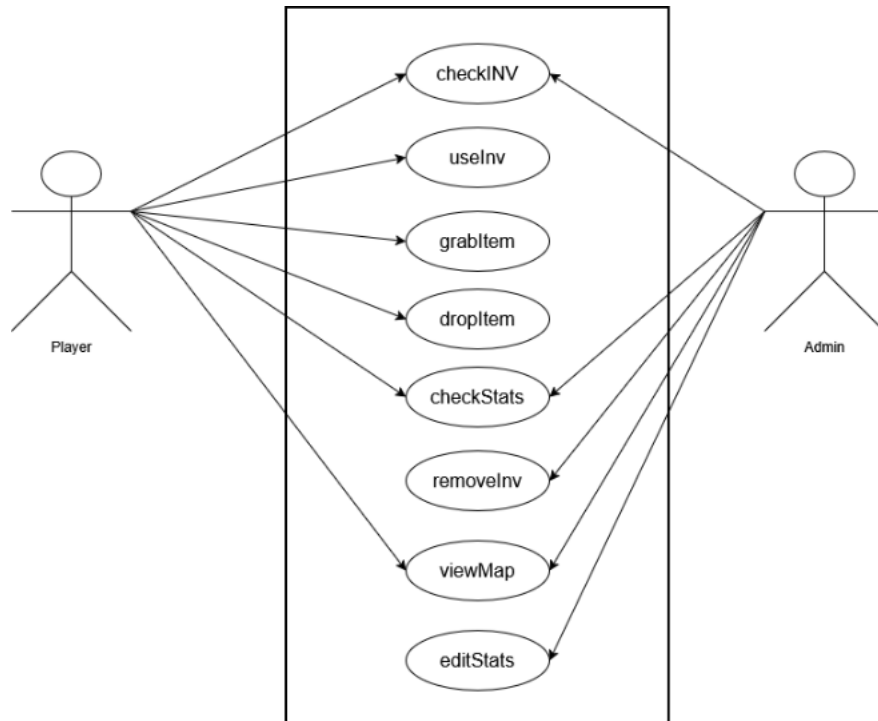
For the puzzle, you will have a textbox prompting you to guess a word. When you make a guess, if the guess contains a correct letter, the text above the guessing box will display what letter in what position is correct. Once all letters are correct, you will have completed the puzzle and can press run away to go back to the map.

UML diagram:

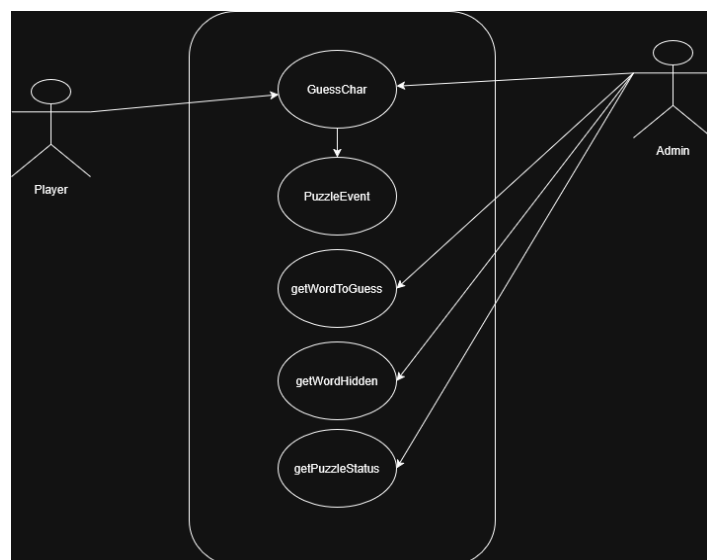


Use case diagram:

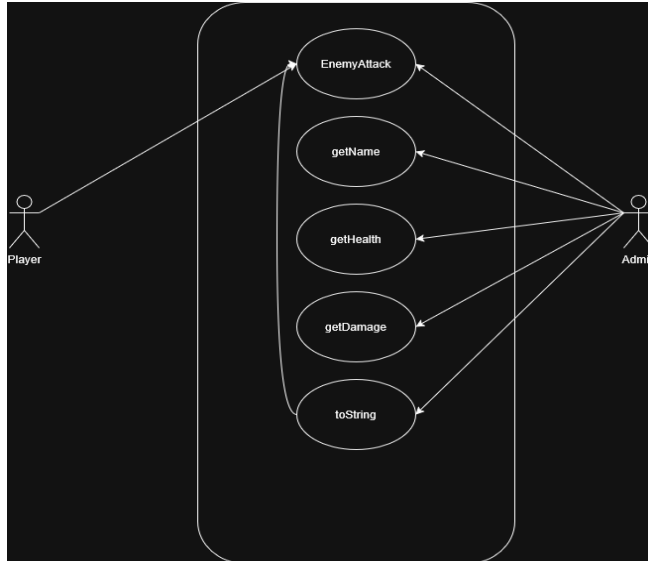
Player



Puzzle System



Enemy System



Testing overview:

We implemented many tests throughout this project. Often times after creating a new method if it did not work but did not throw an error, we could write testing print statement in different parts of the method. If there was an If/else statement or a nested series of statements we found it good to see if those print systems went to the console. Then we could see what parts were targeted. In addition, we would create calls to methods and print those calls to the console to see what was going on, this allowed us to both see what was accessed and what was going on more in depth. Ameer also created the actual test cases for our methods; they can be located in the tests folder in inside src under main.

Known bugs:

- For our project we spent a great deal of time unwinding the serpent of terrible error. Configuring Javafx, learning a new IDE, building a new database with SQLite, maven, lib.xml files pom.xml files, and all other errors that can be thought of we saw, dealt with and worked through. At current there are no major errors that are stopping us from playing, testing and working on the game. However, there are a few things that we ran out of time to fully fix. If the end user switches to the map without logging in and checks their inventory or interacts with the game in a way that targets the back

end, a number of errors will occur, this is because many of our methods call on the database and its associated connection points. With no backend to target as there is no one logged in errors are thrown. I believe we could resolve this by going method by method and adding in statements that check to see if said targeted item is null, then output something like "Please log in." However, we did not have the time to fix this.

- In addition, to this bug we are having a bug with the puzzle obstacle, the confirm button creates two instances of the character guess.
- We are also only able to run the project with a launch from the Maven plug in, we are not sure why we can't run the project from the other run button.

Possible feature additions:

For this we have many different ideas, fixing the bugs we have to start and polishing any aspects we needed to either drop or did not have enough time to implement fully. Post the polishing of our game we want to build out more of our project, adding more enemies, more items and creating a greater amount of depth to the overall project. This can include but is not limited to the following:

- Different enemies with different amounts of health, damage, and player rewards.
- More puzzles
- More items in the shop
- A longer experience for the end player. This way they can feel more rewarded for finishing the game
- We originally wanted to add random encounters for the player, wanderers who could sell items or generic enemies to run into. However, this turned out to be too much for the time we had so we cut that from the version we are submitting.
- Larger playable map
- NPC dialog integrated into the game
- A narrator to guide the player
- Further polish
- Adding sprites for enemies
- Further adding customization and depth to the GUI

Full description of our project files:

Application.java:

Acts as the primary launch point for the project, building the application by launching the other files, starting from saveSelect.fxml and the other SceneBuilder files. The application.java could be thrown into another file but that would only act to further complicate the file structure and what file actually does what. In terms of our application this file acts as the main method.

CurrentPlayerSessionHelperClass.java:

While simple this helper class works to create a current player, the player.java splits the database configuration and this helper class allows us to access the player during their run, if the end user decides to switch to another save this method helps to create a seamless transition for the end user. This calls the player.java and makes an instance of that called currentPlayer, then a simple setter and getter act as the portal to the current session. Now that we are further along in the project, I feel that this could have been added to the Player.java and then called from there but it works well as a separate file, and we have never had any problems with it. When I made it, I was worried about violating naming conventions but in hindsight that would have mattered little.

DatabaseConnection.java:

This chunky file is what connects to our SQLite database, it finds the table with the findsTable method, it finds the primary key with the findsPrimaryKey method. The loadPlayer method takes in the saveId that the end user is currently in, then targets both the table and primary key along with selecting everything from the table that targets it. Once the SQL query has been executed the if statement triggers and returns all of the contents of the database. This is all wrapped up in a try catch to make sure that any errors that occur we know where they occur. The updateMethodForItems could have probably also gone in MapController.java but it works here too. This method works similarly to the loadPlayer method, in fact if we had more time, we could probably find a way to put them together but as it is now, they work well enough. This method is what is responsible for a majority of the methods in MapController.java which themselves tie to many of our backend buttons. Finally, the grabbingInfo method acts as a helper method for the

Enemy.java:

This is the enemy class responsible for the logic behind what the player fights, health, damage and the dialog options behind the interaction. The enemy class also contains an enemyEvent, which handles interaction with the player and the enemy. The player is thrown into a loop in a turn-based system, contained in a loop, in which they must attack or flee. If

player attacks, enemy may be hit or player misses. This is controlled by a random value within a given range. Certain values determine a hit and others a miss. The enemy also can hit or miss and is also determined by a random value in a range.

If a player is hit, their health is affected.

eventDriver.java:

This is a testing main function for the NPC class

GameplayController.java/Launcher.java:

The GameplayController method calls the gameplay.fxml file and will hold backend logic for the players' gameplay options. The Launcher.java file does not do anything at the moment and might be removed.

MapController.java:

This is the backend logic that was being referenced in the DatabaseConnection.java paragraph. All the methods for adding and removing the different items along with the functionality for the inventory viewing logic goes here.

NPC.java:

This is backend logic for NPC interactions this targets the text files that hold the strings of information for interactions. Uses Switch block to determine what file or function to access depending on the NPC type.

If the NPC is a shopkeeper, we decided to create a shop function, containing an array list of items that a user can purchase from. We decided to use a Java record, a class that only requires us to only declare parameter type and name. It serves to be a much simplistic way to contain data and removes the need for a hashMap and the possible walls it may introduce. This choice was made to create pairs of data that may contain similar value. For a shop, this is a valid reason as some items may contain similar prices, where a hashMap may only allow unique values

The shop function is quite simple. A user enters a shop, purchases items, and a transaction occurs. This is managed by a loop and exits when the user feels like they want to exit

Player.java:

This is the player object itself; it holds the objects that are tied to the database and allows the end user to print them to the GUI with the getDisplayToGUI method. The

isPasswordCorrect method checks to see if the password entered matches what is in the database. The player is what is updated as a live session whenever a change is made within the database through the gameplay.

Puzzle.java:

This is one of the obstacles within the game, while we have the enemy class this also acts as something to hinder the player until they are able to solve the puzzle to move forward. For simplicity's sake, we chose to go for a guessing game, much like hangman. Through a loop, the user is presented a "hidden" word to guess.

The puzzle words are gathered from a text file containing a set of word banks. This is then transferred onto an array list, in the case that words are added to the word bank. The word is placed into a String variable, the hidden version into a StringBuilder to allow for correct guesses to be added into the hidden word, while still keeping other letters hidden

SaveSelectController.java:

This file deals with logging in and logging out, the password is stored within the database, which is then connected to the DatabaseConnection, the password then follows this track until it is validated and the player can log in. Once the player has logged in, they are able to access their inventory and all of their stats. Then if they ever want to switch to another save slot, they are able to do so because their session is saved and live updates to the database.

SceneManager.java:

This handles the switching between the different fxml scenes.

ShopController.java:

This file controls the backend logic for the shop and will allow a user to buy and sell options as well as view their inventory similar to how the mapController.java file does

Testing Overview:

Testing was conducted throughout development to verify that the application's major features functioned correctly. The login and save system was tested to ensure that password validation and save selection worked as intended. Database testing confirmed

that player data, including health, inventory, currency, and obstacle completion progress, could be correctly loaded and updated within the SQLite database.

Gameplay testing focused on combat encounters, puzzle completion, inventory management, and scene transitions between the various JavaFX screens. Additional testing was performed on the end-game functionality to ensure that players could only access the completion screen after meeting the required objectives. If the requirements were not met, the game correctly displayed feedback indicating the remaining objectives. Once completed, the end-game screen successfully displayed the player's final statistics and allowed navigation back to the map or login screen.

Our resources folder:

This folder stores the css and .fxml files that are used to create the GUI.

Our Database:

We used SQLite for this project as to eliminate the need for a server; this helps to create a complete application. The 380DatabaseSQLiteFinal.db is our database, each table is fit for a save and each table has the ability to be live updated from the GUI with the player session previously mentioned.